

VAULT-DECK MK-I

IoT-Based Mechatronic Telemetry Station & Cyberdeck

Developer: Emre Göral

Date: June 2026

1. SUMMARY

The Vault-Deck MK-I is an integrated embedded system designed around the "hand-held cyberdeck" concept. It processes environmental sensor data in real-time, provides high-speed telemetry over a local network (WLAN), and communicates seamlessly with cloud-based API services.

The primary engineering objective of this project is to unify mechanical design, hardware electronics, and digital signal processing disciplines within a single, holdable, rigid enclosure. Moving beyond simply reading raw environmental data, the system actively purifies these signals through digital software filters and transmits them to a remote monitoring station (MATLAB) for real-time visual analysis. It also has a mini-game coded inside.

Featuring military-tactical radio (does not play sound, but can control the device that does) and retro-futuristic aesthetic lines, the device is powered by a 2S LiPo battery and built upon a custom power distribution board (Perfboard) architecture to ensure stable voltage regulation and sensor reliability.

Core System Capabilities:

- **Mechanical Hardware & Chassis:** A modular 3D-printed enclosure engineered in SolidWorks featuring a 3-layer architecture (Front Faceplate, Mid-Frame Chassis, and Backplate), meticulously designed in accordance with industrial tolerance rules and 3D printing parameters.
- **Firmware & Interface (Vault-OS):** A 7-tab interactive graphical user interface developed in C++ on an ESP32 microcontroller, utilizing a tab-based state-machine architecture with millis()-based timing for telemetry, API polling, and game frames.

- **Cloud Integration (API):** Direct hardware-level OAuth 2.0 authentication enabling cross-device media control via the Spotify Web API, alongside live weather synchronization using the OpenWeather API.
- **Telemetry & Signal Processing (DSP):** Reduction of AC line noise and hardware ghosting from acoustic (KY-037) and optical (LDR) sensor data using Exponential Moving Average (EMA) and Peak Detection digital filters. The purified data is sent at a high-speed rate of 10 packets per second via UDP to a custom MATLAB oscilloscope dashboard.



Figure 1: Cyberdeck final view

2. MECHANICAL ARCHITECTURE & HARDWARE DESIGN

The physical enclosure of the Vault-Deck MK-I was entirely engineered in SolidWorks. Rather than utilizing a hollow, single-body shell, the system was developed using an industrial "Mid-Frame Architecture" divided into three distinct layers. This modular approach isolates the aesthetic interface, the computational logic, and the power delivery systems.

2.1. The 3-Layer "Sandwich" Architecture

Layer 1: Front Faceplate (User Interface): Designed primarily as an aesthetic armor, this layer houses the chamfered window for the 2.4" SPI TFT display. To account for physical variations, the screen mounts utilize slotted holes (on the right side), allowing for 0.5 mm positional calibration. A custom "volcanic" protrusion was modeled on the lower section and on the upper section to seamlessly absorb the depth of the potentiometer and the screen without thickening the entire chassis.

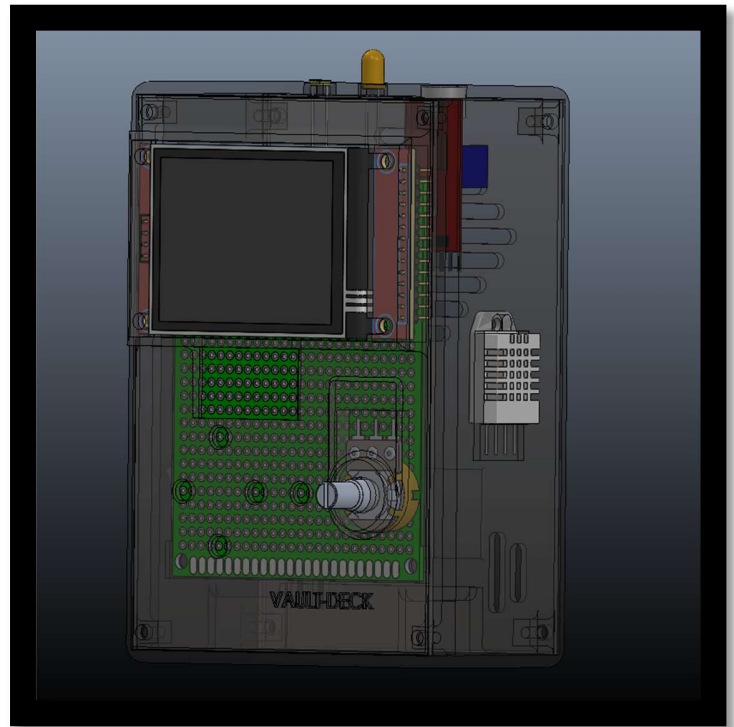


Figure 2 and 3: Solidworks designs front view and transparent view

Layer 2: Mid-Frame Chassis (The Spine): Serving as the main load-bearing structure, this central layer secures the 7x9 cm double-sided perfboard. It acts as the stable foundation for the ESP32 microcontroller and the centralized wiring harness. By isolating the computing components on this rigid mid-frame rather than the removable back cover, the system ensures that delicate signal pathways and sensor connections remain completely undisturbed during battery replacements or hardware inspections.

Layer 3: Backplate (Closure & Thermodynamics): The deepest section of the device is dedicated to the heavy 2S LiPo battery (mounted vertically on the ground in a specific tub) and the LM2596 step-down converter. Based on thermodynamic principles, ventilation channels were strategically cut into the upper half of this back cover, with smaller intake channels at the bottom. This allows

cooler air to enter from the bottom and naturally push the rising hot air (generated by the buck converter) out through the top.

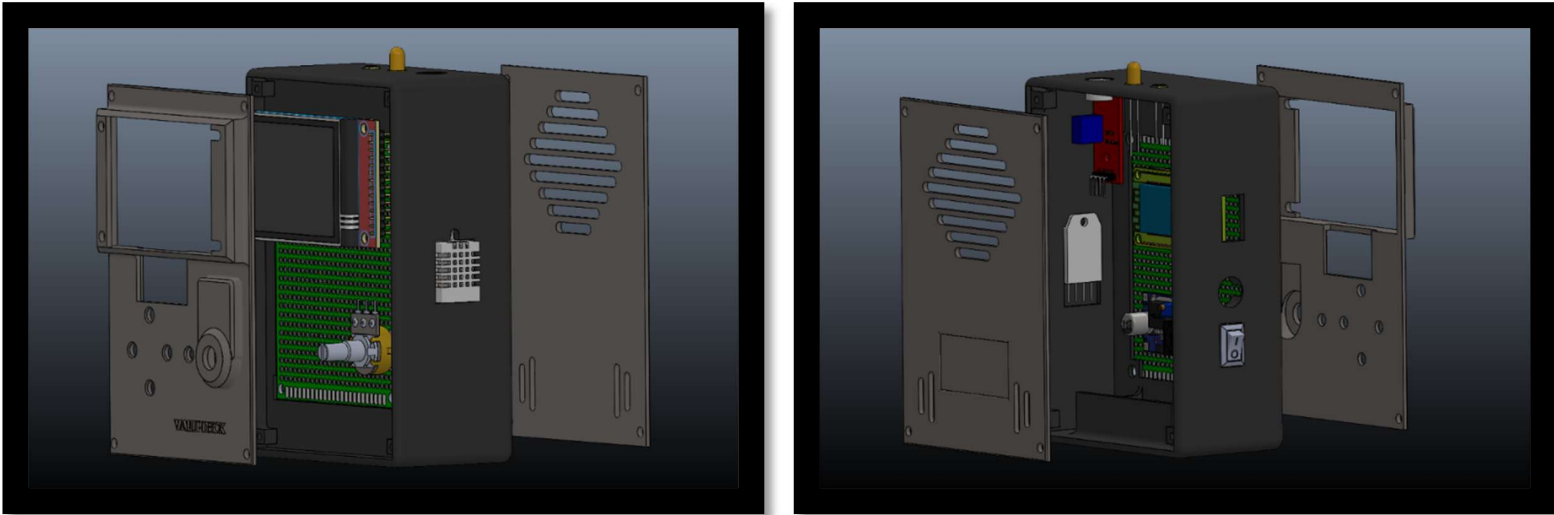


Figure 4 and 5: Solidworks designs semi exploded views

2.2. Design for Additive Manufacturing

To ensure structural integrity during 3D printing, strict geometric rules and tolerances were applied throughout the SolidWorks modeling phase:

Nozzle Multiplier Rule: All wall thicknesses were mathematically defined as multiples of 0.4 mm (the standard 3D printer nozzle diameter). This prevents printer micro-jitter and eliminates weak, under-extruded infill gaps.

Extrusion Tolerances: Circular cutouts for external components (e.g., the 7 mm potentiometer shaft) were modeled with a +0.2 mm to +0.3 mm offset to compensate for thermal plastic shrinkage during the extrusion process.

Torque-Resistant Standoffs: Internal screw standoffs for M2 screws (2.0 mm inner hole) were designed with a minimum outer diameter of 5 mm. This heavy-duty wall thickness prevents the plastic columns from splitting under the torque of assembly screws.

2.3. Power Distribution & Custom PCB Layout

The hardware relies on a highly stabilized, multi-stage power delivery network to prevent voltage drops during Wi-Fi transmission or sensor polling.

Power Routing: Power flows from an external 8.4V DC Jack or the internal 2S LiPo battery, passes through a heavy-duty KCD11 rocker switch, and enters the LM2596 buck converter. The converter is manually trimmed to output a pristine 5.0V to the ESP32 microcontroller.

"Power-Bus" Logic: To eliminate cable-spaghetti within the chassis, a centralized power distribution architecture was implemented on the perfboard. Two continuous horizontal solder bridges were created to act as the Main 3.3V Line and Main GND Line. All peripheral sensors (DHT22, LDR, KY-037) draw their power directly from these robust, low-resistance parallel rails, keeping the data pathways clean and organized. **For example:** the tactile buttons are also using the very same solder bridge.

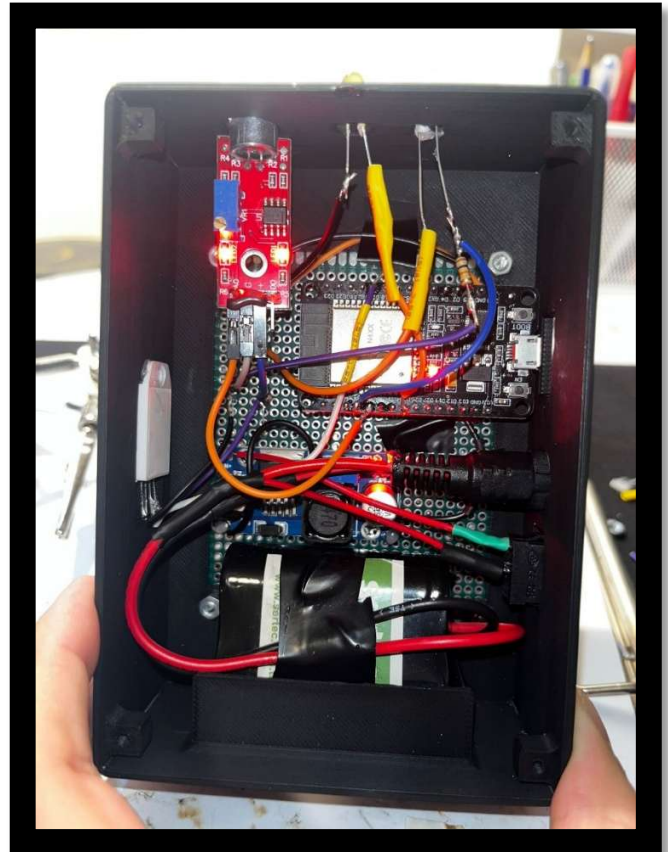
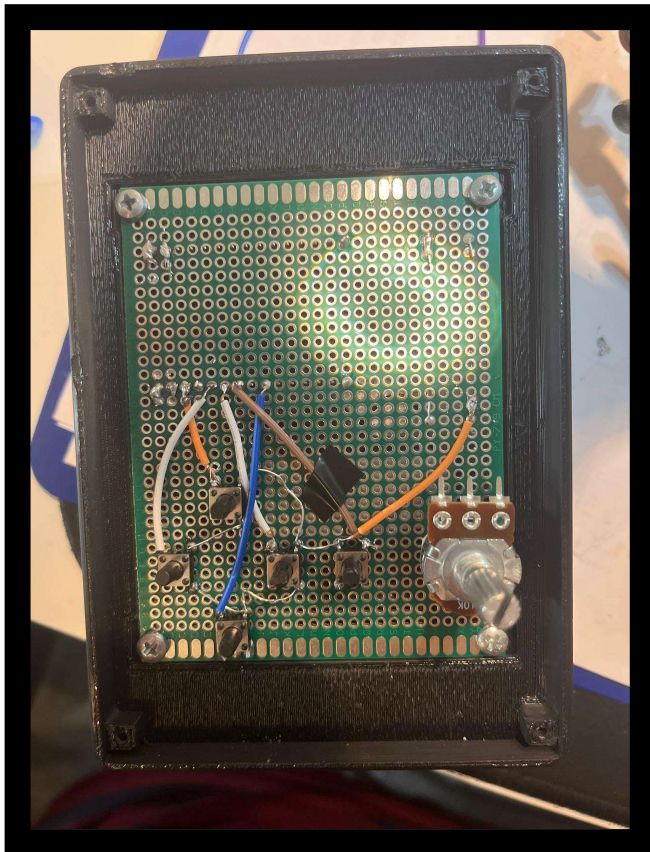


Figure 6 and 7: Inside the frontplate and the backplate

3. FIRMWARE, API INTEGRATION & SIGNAL PROCESSING

The software architecture of the Vault-Deck MK-I is built to coordinate multiple tasks: driving a display, polling hardware sensors, communicating with cloud APIs, and sending telemetry data. Each tab runs its own update routine, keeping the interface responsive.

3.1. Development Environment & State-Machine Architecture

The firmware was developed in C++ using PlatformIO. To ensure a reproducible and professional codebase, hardware-specific display configurations (such as SPI pins, ILI9341_DRIVER, and

clock speeds) were injected directly into the compiler via **platformio.ini** build flags (**build_flags = -D USER_SETUP_LOADED=1 ...**).

This "zero-touch configuration" approach dynamically configures the **TFT_eSPI** library, keeping the core library files unmodified and the project highly portable.

The Vault-OS interface operates on a tab-based **State-Machine architecture**.

- Instead of relying on long, system-halting delays, the core loop utilizes `millis()` timers for telemetry, API polling, weather refresh, and game frames; only short delays remain for button debouncing and display refresh.
- The UI consists of 7 distinct, interactive tabs: System Overview (STAT), Environment Radar (ENV), Acoustic Radar (WAVE), Radio Terminal (RDO), MATLAB Telemetry (DIAG), Tactical Flashlight (LGHT), and an Orbit Defense mini-game (GAME).
- Navigation is handled through a custom tactile D-pad utilizing the ESP32's internal pull-up resistors (`INPUT_PULLUP`) to minimize external hardware footprint.

To complement the system's interactive capabilities, specific utility and diagnostic modules are seamlessly integrated into the Vault-OS firmware architecture. **The manual 10K potentiometer functions as a tactical flashlight utility**, directly regulating the pulse-width modulation (PWM) brightness of the chassis-mounted yellow LED while simultaneously projecting a responsive, automotive-style gauge on the TFT display to visualize real-time intensity. Live environmental feedback is further expanded within the **Acoustic Radar (WAVE) tab**, which renders a dynamically shifting waveform that fluctuates according to external acoustic data captured by the microphone. Finally, essential network and hardware metrics are centralized in the **System Overview (STAT) tab**, providing critical runtime telemetry including the station's assigned local IP address, the active power source designation, and continuous system uptime.

3.2. Cloud Connectivity & RESTful APIs

At startup, the device offers a dual-boot sequence, allowing the user to initialize in a localized Offline Mode or connect to a WLAN for cloud integration.

OpenWeather API: The system executes HTTP GET requests to fetch real-time outdoor temperature and humidity data for Istanbul, parsing the JSON payload via `ArduinoJson` to display a comparative analysis against the onboard DHT22 indoor sensor data.



Figure 8: ENV tab

Spotify Web API (OAuth 2.0): Utilizing an HTTPS client (WiFiClientSecure, with certificate validation disabled in this prototype), the terminal authenticates with Spotify's servers to act as a remote media cyberdeck. It retrieves the currently playing track, artist name, and duration. User can also control whether if they want to skip or stop/resume.



Figure 9: RADIO (Spotify) tab

Client-Side Prediction Algorithm: To prevent API rate-limiting (polling the server too frequently), a smart simulation algorithm was coded. The ESP32 calculates the track's progress locally based on its internal clock ($\text{currentSimulatedProgress} += (\text{millis}() - \text{lastApiSyncTime})$), creating a smooth, real-time progress bar without spamming the Spotify servers.

3.3. Digital Signal Processing (DSP) & High-Speed Telemetry

The true engineering depth of the project lies in how it handles raw analog signals before sending them to the MATLAB dashboard. Raw sensor data in mechatronics is notoriously noisy; therefore, digital filters were implemented in the `sendTelemetry()` pipeline before the data is packaged into UDP packets at a rate of 10 Hz. Telemetry is sent while the DIAG tab is active.

Hardware Ghosting Mitigation: When rapidly switching between analog pins on the ESP32, residual voltage in the ADC capacitor can cause "ghosting" (signals bleeding into each other). To solve this, the firmware performs a "dummy read" on the LDR pin, waits 2 milliseconds to let the voltage settle, and then takes the actual reading.

Acoustic Peak Detection (Microphone Gain): Audio waves move faster than the 100ms telemetry cycle. Instead of taking a single snapshot, the software rapidly samples the KY-037 analog pin 50 times in a rapid loop (200 microseconds apart) to find the true amplitude ($\text{micMax} - \text{micMin}$). A digital software gain (multiplier of 15x) is then applied to amplify the acoustic data into highly visible, seismic spikes on the MATLAB oscilloscope.

AC Flicker Noise Reduction (EMA Filter): The LDR sensor is sensitive enough to pick up the invisible 50Hz alternating current (AC) flicker of room lighting, causing a jagged "sawtooth" wave on the data plot. An Exponential Moving Average (EMA) filter was coded: $\text{smoothedLDR} = (\text{smoothedLDR} * 0.85) + (\text{rawLDR} * 0.15)$. This mathematical "ironing" greatly reduces the AC noise, resulting in a stable optical telemetry curve.

4. CONCLUSION & SYSTEM PERFORMANCE

The Vault-Deck MK-I successfully fulfills its primary design objective as a handheld field diagnostic and control terminal, seamlessly blending retro-futuristic aesthetics with modern IoT capabilities. By integrating mechanical CAD modeling, custom hardware power distribution, and advanced embedded software, the project transitioned from a conceptual prototype to a highly stable, field-ready cyberdeck.

4.1. Hardware and Thermal Stability

The implementation of the 3-layer "Mid-Frame" architecture proved highly effective. The physical separation of the 2S LiPo battery and the LM2596 buck converter into the bottom "Battery Tub" ensured that the ESP32 microcontroller and sensitive sensor pathways remained thermally isolated and physically secure. The custom Power-Bus Perfboard successfully eliminated wiring clutter, providing a pristine, uninterrupted 5.0V/3.3V power delivery network that prevented voltage drops even during heavy Wi-Fi and display usage.

4.2. Telemetry and DSP Efficiency

The most significant achievement of the system is its real-time data transmission reliability. The UDP streaming pipeline operates at a steady 10 Hz cycle with near-zero packet loss over the local network. Furthermore, the application of Digital Signal Processing (DSP), specifically the Peak Detection algorithm for acoustics and the Exponential Moving Average (EMA) filter for optics, successfully reduced hardware ghosting and 50Hz AC line noise. This resulted in smooth, professional-grade waveforms on the remote MATLAB oscilloscope interface.

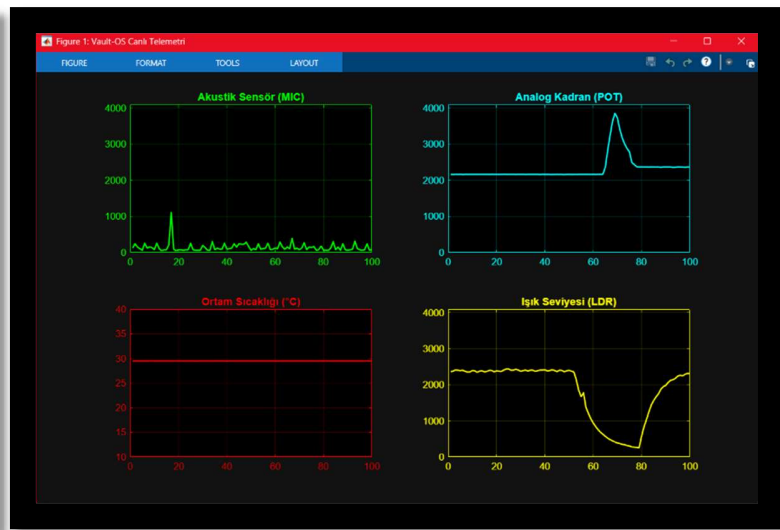


Figure 10 and 11: MATLAB graphs that are extracted from the Cyberdeck

4.3. Final Verdict

Ultimately, the Vault-Deck MK-I demonstrates a comprehensive mastery of mechatronic system design. It proves that complex subsystems, including multi-tab C++ GUI management, cloud API authentication (OAuth 2.0), localized sensor fusion, and high-speed telemetry, can run efficiently

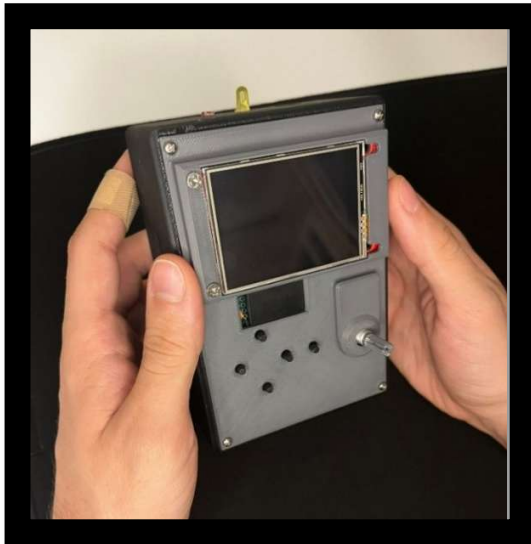


Figure 12 and 13: Holding cyberdeck in my hand and playing the mini game together on a single, battery-powered microcontroller housed within a precision-engineered 3D-printed enclosure.

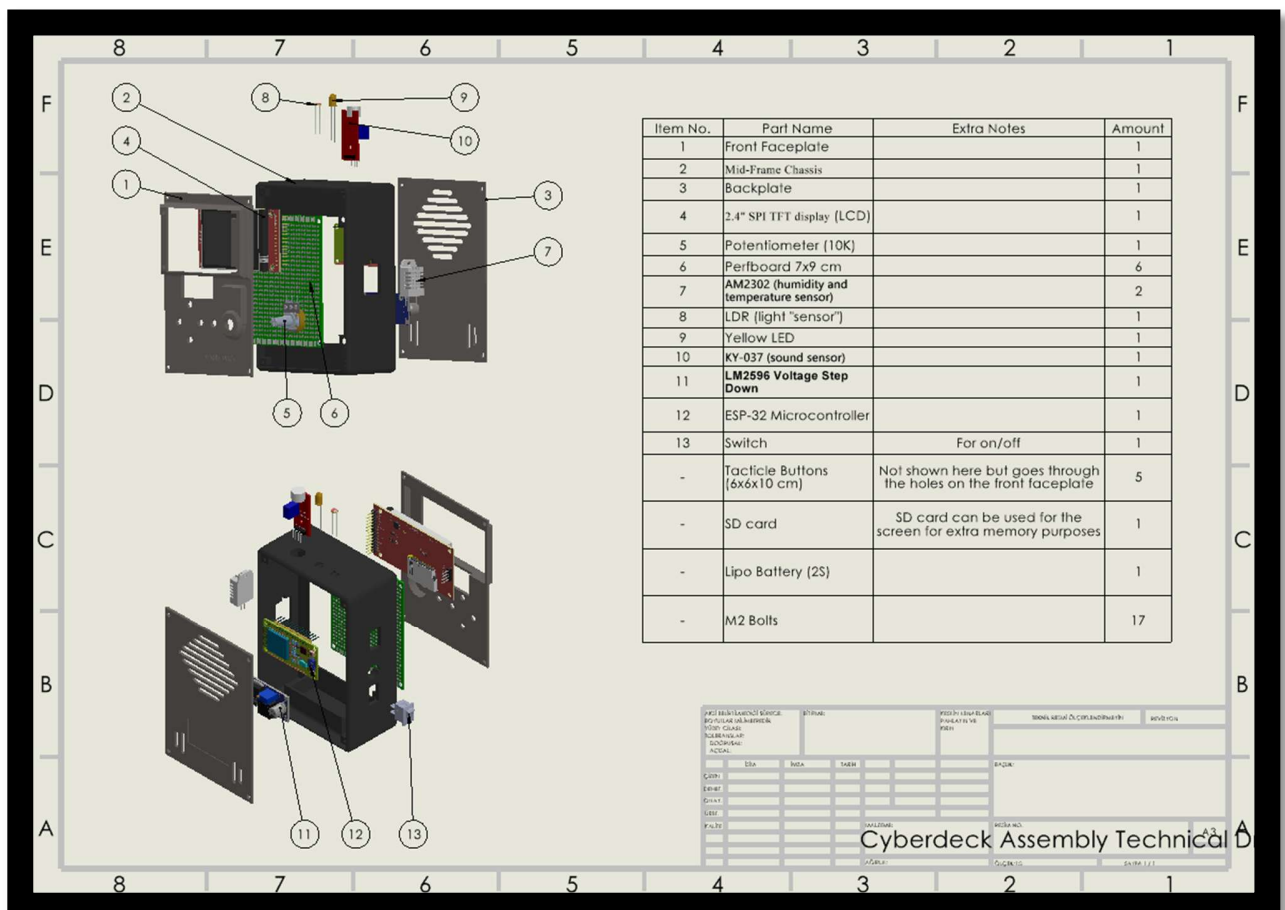


Figure 14: EXPLODED views and the BOM